

CO2 - AT2 - DEBUGGING TASK

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence.

Sol:

Perceptron Fails to Converge

Possible Causes

- Learning rate is too high or too low.
- Incorrect weight initialization.
- Bias is missing or updated incorrectly.
- Wrong implementation of the weight update rule.

Debugging Steps

1. Verify the dataset is linearly separable.
2. Check whether weights and bias are updating correctly.
3. Print weight values after each iteration.
4. Test the algorithm on a small dataset.
5. Compare output with expected results.

Optimization Techniques

- Use a proper learning rate (e.g., 0.01 or 0.1).
- Initialize weights randomly with small values.
- Update bias correctly.
- Normalize input data.
- Stop training when no errors remain.


```
▶ from sklearn.linear_model import Perceptron
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score

# Generate linearly separable dataset
X, y = make_classification(n_samples=200,
                           n_features=2,
                           n_redundant=0,
                           n_clusters_per_class=1,
                           class_sep=2.5,
                           random_state=42)

model = Perceptron(max_iter=1000,
                   eta0=0.1,
                   random_state=42)

model.fit(X, y)

pred = model.predict(X)

print("Training Accuracy:", accuracy_score(y, pred))
print("Weights:", model.coef_)
print("Bias:", model.intercept_)
print("Iterations:", model.n_iter_)
```

```
... Training Accuracy: 0.99
Weights: [[ 0.33176429 -0.24214932]]
Bias: [0.4]
Iterations: 13
```

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance.

Sol:

MLP with Backpropagation Shows Slow Convergence

Possible Causes

- Vanishing gradients.
- Poor activation functions (Sigmoid).
- Bad weight initialization.
- Learning rate too high or too low.

Debugging Steps

- Monitor training and validation loss.
- Check gradient values.
- Verify backpropagation calculations.
- Test different activation functions.
- Observe weight updates.

Optimization Techniques

- Use ReLU activation.
- Apply Xavier or He initialization.
- Normalize input data.
- Use Adam optimizer.
- Use adaptive learning rates.

```
▶ from sklearn.datasets import load_iris
  from sklearn.model_selection import train_test_split
  from sklearn.neural_network import MLPClassifier
  from sklearn.metrics import accuracy_score

iris = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    iris.data,
    iris.target,
    test_size=0.2,
    random_state=42)

model = MLPClassifier(hidden_layer_sizes=(50,),
                      activation='relu',
                      solver='adam',
                      learning_rate_init=0.001,
                      max_iter=500)

model.fit(X_train, y_train)

prediction = model.predict(X_test)

print("Accuracy =", accuracy_score(y_test, prediction))
print("Loss =", model.loss_)
```

```
... Accuracy = 1.0
   Loss = 0.12682534288227895
```

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed.

Sol:

Genetic Algorithm Gives Poor Solutions

Possible Causes

- Incorrect chromosome encoding.
- Poor fitness function.
- Premature convergence.
- Low population diversity.

Debugging Steps

1. Verify chromosome representation.
2. Check fitness calculations.
3. Monitor mutation and crossover.
4. Observe population diversity.
5. Run multiple experiments.

Optimization Techniques

- Increase mutation rate slightly.
- Use tournament selection.
- Apply elitism.
- Increase population size.
- Improve crossover strategy.

```
import random

def fitness(x):
    return x * x

population = [random.randint(20,31) for i in range(10)]

for generation in range(20):

    population.sort(key=fitness, reverse=True)

    print("Generation:", generation,
          "Best:", population[0],
          "Fitness:", fitness(population[0]))

    parent1 = population[0]
    parent2 = population[1]

    child = (parent1 + parent2)//2

    if random.random() < 0.2:
        child += random.randint(-1,1)

    population[-1] = child
```

```
... Generation: 0 Best: 30 Fitness: 900
Generation: 1 Best: 30 Fitness: 900
Generation: 2 Best: 31 Fitness: 961
Generation: 3 Best: 31 Fitness: 961
Generation: 4 Best: 31 Fitness: 961
Generation: 5 Best: 31 Fitness: 961
Generation: 6 Best: 31 Fitness: 961
Generation: 7 Best: 31 Fitness: 961
Generation: 8 Best: 31 Fitness: 961
Generation: 9 Best: 31 Fitness: 961
Generation: 10 Best: 31 Fitness: 961
Generation: 11 Best: 31 Fitness: 961
Generation: 12 Best: 31 Fitness: 961
Generation: 13 Best: 31 Fitness: 961
Generation: 14 Best: 31 Fitness: 961
Generation: 15 Best: 31 Fitness: 961
Generation: 16 Best: 31 Fitness: 961
Generation: 17 Best: 31 Fitness: 961
Generation: 18 Best: 31 Fitness: 961
Generation: 19 Best: 31 Fitness: 961
```

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability.

Sol:

Deep Neural Network + Genetic Programming Overfits

Possible Causes

- Very complex model.
- Too many parameters.
- Small training dataset.
- Long training time.

Debugging Steps

1. Compare training and testing accuracy.
2. Monitor validation loss.
3. Check model size.
4. Inspect training data quality.
5. Measure computational time.

Optimization Techniques

- Apply Dropout.
- Use L1/L2 Regularization.
- Prune unnecessary neurons.
- Use Early Stopping.
- Reduce model complexity.


```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
```

```
model = Sequential()
```

```
model.add(Dense(64,
                activation='relu',
                input_shape=(20,)))
```

```
model.add(Dropout(0.5))
```

```
model.add(Dense(32,
                activation='relu'))
```

```
model.add(Dense(1,
                activation='sigmoid'))
```

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

```
model.summary()
```

```
... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_2 (Dense)	(None, 64)	1,344
dropout (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 32)	2,080
dense_4 (Dense)	(None, 1)	33

```
Total params: 3,457 (13.50 KB)
```

```
Trainable params: 3,457 (13.50 KB)
```

```
Non-trainable params: 0 (0.00 B)
```

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

Sol:

Decision Tree Overfits

Possible Causes

- Tree is too deep.
- Noisy data.
- Too many features.
- Small leaf nodes.

Debugging Steps

1. Compare train and test accuracy.
2. Visualize tree depth.
3. Check noisy features.
4. Perform cross-validation.
5. Remove irrelevant features.

Optimization Techniques

- Use pruning.
- Limit tree depth.
- Increase minimum samples per leaf.
- Perform feature selection.
- Use cross-validation.

```
▶ from sklearn.datasets import load_iris
   from sklearn.tree import DecisionTreeClassifier
   from sklearn.model_selection import train_test_split
   from sklearn.metrics import accuracy_score

   iris = load_iris()

   x_train,x_test,y_train,y_test = train_test_split(
       iris.data,
       iris.target,
       test_size=0.2,
       random_state=42
   )

   tree = DecisionTreeClassifier(
       max_depth=3,
       random_state=42
   )

   tree.fit(x_train,y_train)

   prediction = tree.predict(x_test)

   print("Accuracy =",accuracy_score(y_test,prediction))
```

```
... Accuracy = 1.0
```

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy.

Sol:

CNN Produces Unstable Results

Possible Causes

- Poor data augmentation.
- Vanishing gradients.
- Batch normalization errors.
- Learning rate issues.

Debugging Steps

1. Check input images.
2. Monitor training loss.
3. Verify batch normalization.
4. Inspect gradients.
5. Compare augmented images.

Optimization Techniques

- Apply Batch Normalization.
- Use ReLU activation.
- Tune learning rate.
- Improve data augmentation.
- Use Adam optimizer.

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

```
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

```
X_train = X_train.reshape(-1, 28, 28, 1) / 255
```

```
X_test = X_test.reshape(-1, 28, 28, 1) / 255
```

```
model = Sequential()
```

```
model.add(Conv2D(32, (3, 3),
                 activation='relu',
                 input_shape=(28, 28, 1)))
```

```
model.add(MaxPooling2D())
```

```
model.add(Flatten())
```

```
model.add(Dense(64,
                 activation='relu'))
```

```
model.add(Dense(10,
                 activation='softmax'))
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
model.fit(X_train,
          y_train,
          epochs=3,
          validation_data=(X_test, y_test))
```

*** Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 — 1s 0us/step

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Epoch 1/3

1875/1875 — 33s 17ms/step - accuracy: 0.9513 - loss: 0.1683 - val_accuracy: 0.9781 - val_loss: 0.0678

Epoch 2/3

1875/1875 — 36s 19ms/step - accuracy: 0.9812 - loss: 0.0607 - val_accuracy: 0.9837 - val_loss: 0.0493

Epoch 3/3

1740/1875 — 2s 18ms/step - accuracy: 0.9878 - loss: 0.0402

7. A reinforcement learning agent fails to learn an optimal policy in a simulated environment. Investigate causes such as poor reward design, insufficient exploration, or incorrect state representation. Explain debugging methods and recommend improvements like reward shaping, exploration strategies, and tuning hyperparameters.

Sol:

Reinforcement Learning Agent Fails to Learn

Possible Causes

- Poor reward function.
- Insufficient exploration.
- Incorrect state representation.
- Wrong hyperparameters.

Debugging Steps

1. Verify reward values.
2. Observe agent actions.
3. Check state representation.
4. Monitor learning curve.
5. Test different environments.

Optimization Techniques

- Reward shaping.
- ϵ -Greedy exploration.
- Better state representation.
- Tune learning rate.
- Increase training episodes.

```
import gym
import numpy as np

env = gym.make("FrozenLake-v1", is_slippery=False)

Q = np.zeros((16,4))

print("Initial Q Table")
print(Q)
```

```
... Initial Q Table
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]
```

Gym has been unmaintained since 2022 and does not support NumPy 2.0 amongst other critical functionality.
Please upgrade to Gymnasium, the maintained drop-in replacement of Gym, or contact the authors of your software and request that they upgrade.

8. A clustering algorithm like K-means produces inconsistent cluster assignments for similar datasets. Analyze possible reasons such as poor initialization, incorrect number of clusters, or sensitivity to outliers. Propose debugging approaches and optimization strategies like K-means++, normalization, and silhouette analysis.

Sol:

K-Means Gives Different Clusters

Possible Causes

- Random initialization.
- Wrong number of clusters.
- Outliers.
- Different feature scales.

Debugging Steps

1. Check initial centroids.
2. Normalize data.
3. Detect outliers.
4. Compare cluster assignments.
5. Calculate silhouette score.

Optimization Techniques

- Use K-Means++ initialization.
- Normalize features.
- Remove outliers.
- Choose optimal K using Elbow Method.
- Validate using Silhouette Analysis.


```
▶ from sklearn.datasets import load_iris  
from sklearn.cluster import KMeans
```

```
iris=load_iris()
```

```
model=KMeans(  
    n_clusters=3,  
    init='k-means++',  
    random_state=42  
)
```

```
model.fit(iris.data)
```

```
print("Cluster Centers")
```

```
print(model.cluster_centers_)
```

```
... Cluster Centers
```

```
[[6.85384615  3.07692308  5.71538462  2.05384615]  
 [5.006        3.428        1.462        0.246       ]  
 [5.88360656  2.74098361  4.38852459  1.43442623]]
```

9. A support vector machine (SVM) model underperforms on a nonlinear dataset. Identify issues such as incorrect kernel choice, improper parameter tuning, or scaling problems. Suggest debugging techniques and optimization methods including kernel selection, hyperparameter tuning, and feature scaling.

Sol:

SVM Performs Poorly

Possible Causes

- Wrong kernel.
- Poor parameter values.
- Features not scaled.
- Incorrect C or Gamma.

Debugging Steps

1. Scale input features.
2. Test different kernels.
3. Tune C and Gamma.
4. Check classification accuracy.
5. Perform cross-validation.

Optimization Techniques

- Use RBF kernel.
- Feature scaling.
- Grid Search.
- Cross-validation.
- Hyperparameter tuning.

```
▶ from sklearn.datasets import load_iris
  from sklearn.model_selection import train_test_split
  from sklearn.preprocessing import StandardScaler
  from sklearn.svm import SVC
  from sklearn.metrics import accuracy_score

  iris=load_iris()

  X_train,X_test,y_train,y_test=train_test_split(
      iris.data,
      iris.target,
      test_size=0.2,
      random_state=42
  )

  scaler=StandardScaler()

  X_train=scaler.fit_transform(X_train)
  X_test=scaler.transform(X_test)

  model=SVC(kernel='rbf')

  model.fit(X_train,y_train)

  prediction=model.predict(X_test)

  print("Accuracy =",accuracy_score(y_test,prediction))
```

```
*** Accuracy = 1.0
```

10. A natural language processing (NLP) model using recurrent neural networks (RNNs) suffers from poor long-term dependency learning. Examine causes such as vanishing gradients and limited memory capacity. Describe debugging steps and propose solutions like LSTM/GRU units, attention mechanisms, and improved training strategies.

Sol:

RNN Cannot Learn Long-Term Dependencies

Possible Causes

- Vanishing gradients.
- Limited memory.
- Long sequences.
- Poor initialization.

Debugging Steps

1. Monitor gradients.
2. Observe training loss.
3. Test shorter sequences.
4. Compare hidden states.
5. Verify data preprocessing.

Optimization Techniques

- Use LSTM or GRU.
- Add Attention Mechanism.
- Gradient Clipping.
- Better weight initialization.
- Use Adam optimizer.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

model = Sequential()

model.add(Embedding(5000, 64))

model.add(LSTM(64))

model.add(Dense(1,
                activation='sigmoid'))

model.summary()
```

... Model: "sequential_4"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	?	0 (unbuilt)
lstm_1 (LSTM)	?	0 (unbuilt)
dense_8 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)
Non-trainable params: 0 (0.00 B)